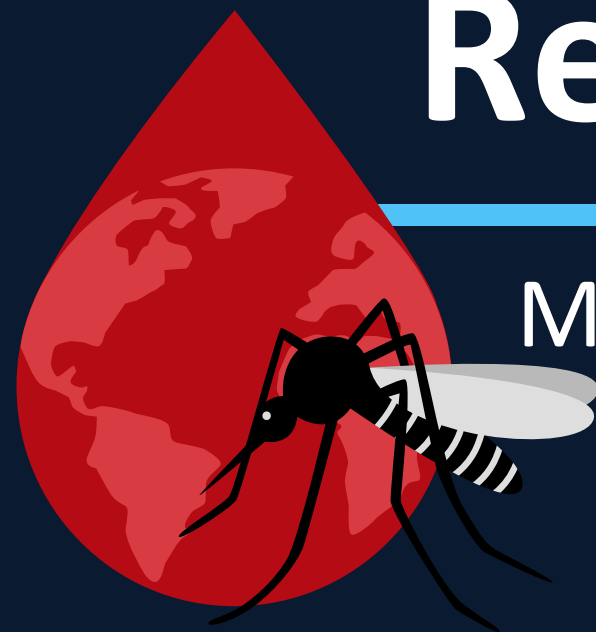


Reto Detección de Malaria



Modelación computacional de sistemas eléctricos

Miranda Henry A01642601

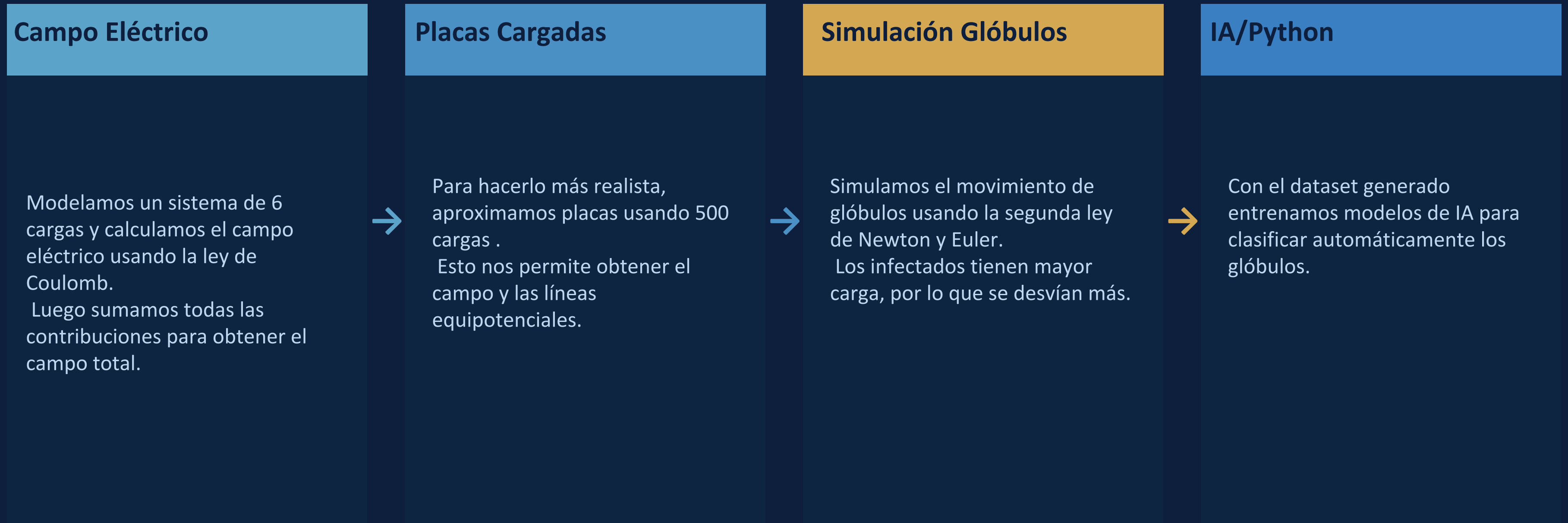
Paula Ferro A01642683

Álvaro Alcaraz A01648252

Osvaldo Sierra A01648528

Kevin Liarte A01649074

Arquitectura del Proyecto



MATLAB · Parte 1: Configuración de 6 Cargas Puntuales

Ley de Coulomb para cada carga → Superposición vectorial

¿Qué hace?

1. Definimos un sistema de 6 cargas distribuidas en dos columnas.
2. Calculamos el campo eléctrico en cada punto de una malla utilizando la ley de Coulomb.
3. Sumamos las contribuciones individuales de cada carga para obtener el campo total.
4. Normalizamos el campo para visualizar únicamente su dirección y lo representamos en una gráfica en 3D.

Expresión del campo eléctrico

$$E = k \cdot q / r^2 \rightarrow ex = (q / r^3) \cdot rx$$

```
% --- Definición de cargas ---
```

```
q1=1; q2=-1; q3=1;  
q4=-1; q5=1; q6=-1;  
d = 0.5;
```

- Definir cargas

```
[x,y] = meshgrid(-1:0.1:1, -1:0.1:1);  
z = zeros(size(x));
```

- Crear espacio donde se evalúa el campo

```
% Posiciones en 2 columnas
```

```
x1=-d; y1=-1; x2=d; y2=-1;  
x3=-d; y3=0; x4=d; y4=0;  
x5=-d; y5=1; x6=d; y6=1;
```

- Ubicar las cargas en dos columnas

```
% --- Campo de q1 ---
```

```
rx1 = x - x1; ry1 = y - y1;  
r1 = sqrt(rx1.^2 + ry1.^2);  
ex1 = (q1./r1.^3) .* rx1;  
ey1 = (q1./r1.^3) .* ry1;
```

- Calcular campo de cada carga
(Se repite para cada carga)

```
% --- Campo total (superposición) ---
```

```
ex = ex1 + ex2 + ex3 + ex4 + ex5 + ex6;  
ey = ey1 + ey2 + ey3 + ey4 + ey5 + ey6;  
ez = zeros(size(ex));
```

- Sumar todos los campos

```
% --- Magnitud y normalización ---
```

```
em = sqrt(ex.^2 + ey.^2 + ez.^2);  
exn = ex./em;  
eyn = ey./em;  
ezn = zeros(size(exn));
```

- Ajustar para ver solo dirección

```
% --- Graficar campo ---
```

```
figure  
hold on  
axis equal  
view(3)  
axis([-1 1 -1 1 -1 1])  
grid on
```

- Crear campo

```
quiver3(x, y, z, exn, eyn, ezn, 'g')
title('Campo con seis cargas')
xlabel('x')
ylabel('y')
zlabel('z')
```

- Graficar campo en 3D

```
hold off
```

```
v = 0.5;
```

```
function vertices = crear_prisma(xc, yc, zc, ancho, largo, v)
vertices = [
-v*ancho+xc, -v*largo+yc, -v+zc;
-v*ancho+xc, v*largo+yc, -v+zc;
v*ancho+xc, v*largo+yc, -v+zc;
v*ancho+xc, -v*largo+yc, -v+zc;
-v*ancho+xc, -v*largo+yc, v+zc;
-v*ancho+xc, v*largo+yc, v+zc;
v*ancho+xc, v*largo+yc, v+zc;
v*ancho+xc, -v*largo+yc, v+zc];
end
```

- Función para generar vértices

```
% --- Parámetros ---
dx = -0.5; dy = 0; dz = 0;
```

- Posición inicial

```
largo_roja = 3;
largo_azul = 3;
ancho = -0.2;
```

- Dimensiones de placas

```
x_roja = -d;
x_azul = d;
```

- Ubicar placas en x

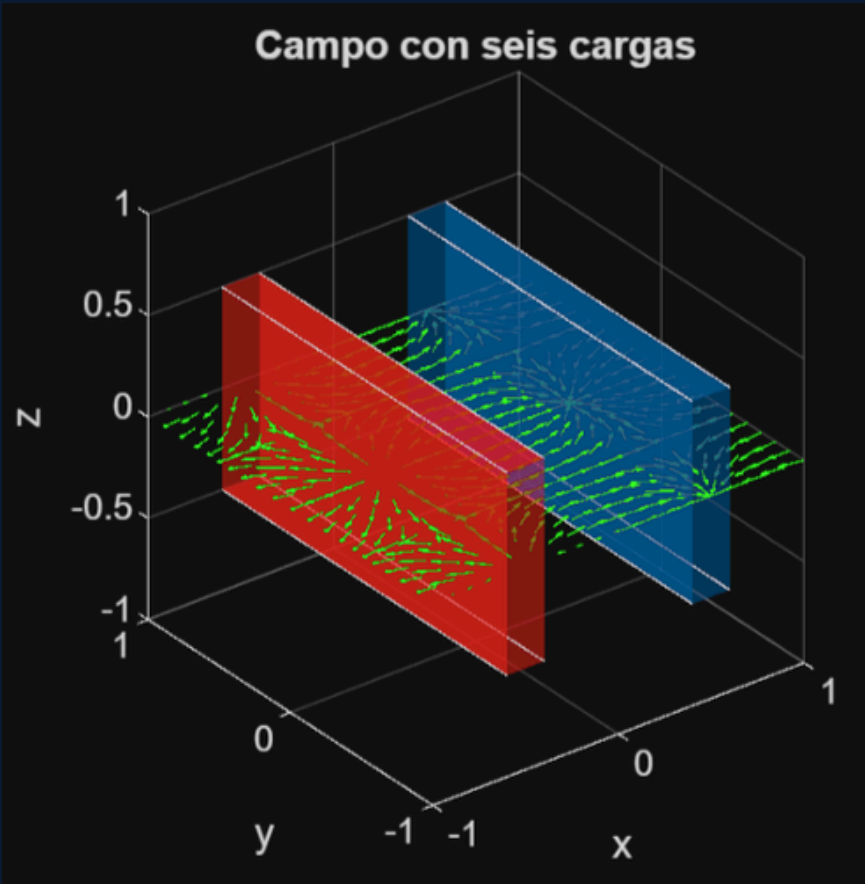
```
% --- Caras del prisma ---
caras = [1 2 3 4;
2 6 7 3;
4 3 7 8;
1 5 8 4;
1 2 6 5;
5 6 7 8];
```

- Definir caras del prisma

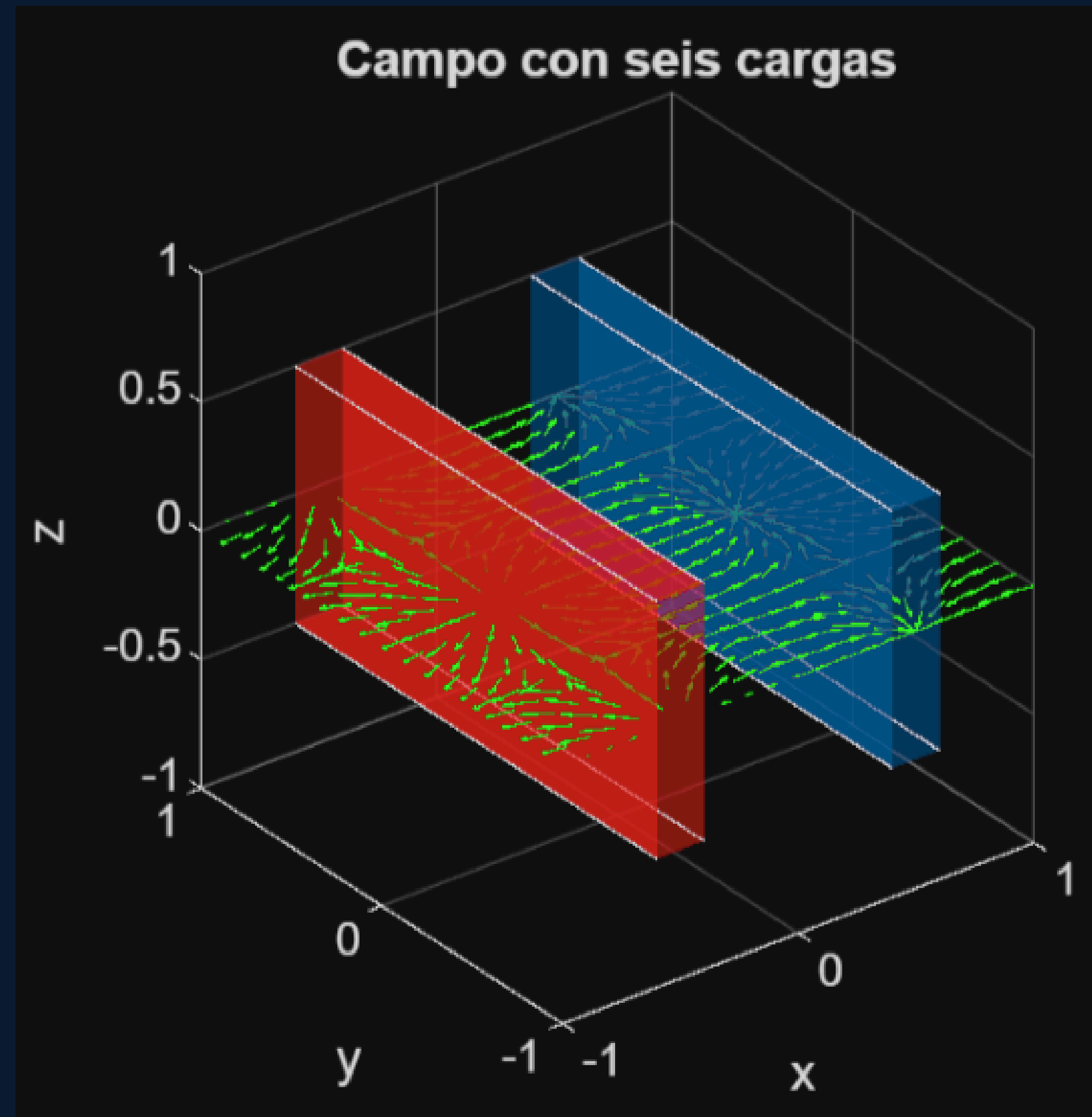
```
% --- Crear placas ---
vertices_roja = crear_prisma(x_roja, dy, dz, ancho, largo_roja, v);
vertices_azul = crear_prisma(x_azul, dy, dz, ancho, largo_azul, v);
```

```
% --- Dibujar placas ---
patch('Faces', caras,'Vertices',
vertices_roja,'FaceColor','r','FaceAlpha',0.5);
patch('Faces', caras,'Vertices',
vertices_azul,'FaceColor','b','FaceAlpha',0.5);
```

- Graficar Prismas



GRÁFICA - Configuración de 6 Cargas Puntuales



MATLAB · Parte 2: Placas Cargadas

Aproximación de distribución continua con 500 cargas puntuales por placa

Idea General

1. En esta segunda parte modelamos placas cargadas.
2. Para eso colocamos 500 cargas por placa.
3. Calculamos el campo eléctrico y el potencial en cada punto de la malla sumando todas las contribuciones individuales.
4. Gradicamos el campo mediante líneas de flujo y el potencial con líneas equipotenciales

Variables

- $k_e = 9 \times 10^9$ (constante de Coulomb)
- q_1, q_2 (cargas de las placas)
- d_{qp}, d_{qn} (carga por punto)
- Cálculo de E_x, E_y y V
- Visualización: campo + equipotenciales

```
% --- Parámetros de placas ---
```

```
d = 1;
```

```
ke = 9e+9;
```

```
q1 = 1e-5;
```

```
q2 = -1e-5;
```

- Definir parámetros del sistema

```
lyp = 500;
```

```
lyn = 500;
```

- Definir cantidad de cargas

```
dqp = q1 / lyp;
```

```
dqn = q2 / lyn;
```

- Calcular la carga de cada punto

```
% --- Crear malla ---
```

```
x_vals = linspace(-2, 2, 80);
```

```
y_vals = linspace(-2, 2, 80);
```

```
[x, y] = meshgrid(x_vals, y_vals);
```

- Crear la malla

```
% --- Inicializar matrices ---
```

```
Ex = zeros(size(x));
```

```
Ey = zeros(size(y));
```

```
V = zeros(size(x));
```

- Inicializar matrices para campo y potencial

```
% --- Coordenadas de las cargas ---
```

```
y_val_p = linspace(-d, d, lyp);
```

```
y_val_n = linspace(-d/4, d/4, lyn);
```

- Distribuir cargas

```
x_val_p = ones(size(y_val_p)) * -0.25;
```

```
x_val_n = ones(size(y_val_n)) * 0.25;
```

```
% --- Calcular campo y potencial ---
for i = 1:length(x_vals)
    for j = 1:length(y_vals)

        px = x(i,j);
        py = y(i,j);

        % --- Placa positiva ---
        for n = 1:lyp
            rx = px - x_val_p(n);
            ry = py - y_val_p(n);
            r = sqrt(rx^2 + ry^2);

            Ex(i,j) = Ex(i,j) + ke * dqp * rx / r^3;
            Ey(i,j) = Ey(i,j) + ke * dqp * ry / r^3;
            V(i,j) = V(i,j) + ke * dqp / r;
        end

        % --- Placa negativa ---
        for m = 1:lyn
            rx = px - x_val_n(m);
            ry = py - y_val_n(m);
            r = sqrt(rx^2 + ry^2);

            Ex(i,j) = Ex(i,j) + ke * dqn * rx / r^3;
            Ey(i,j) = Ey(i,j) + ke * dqn * ry / r^3;
            V(i,j) = V(i,j) + ke * dqn / r;
        end
    end
end
```

- Recorrer cada punto de la malla

- Punto donde se calcula

- Sumar contribución en la placa positiva

- Sumar contribución en la placa negativa

```
% --- Visualizar campo y equipotenciales ---
figure(1)
contour(x, y, V, 20, 'Color', [0.6 0 1])
hold on
streamslice(x, y, Ex, Ey)

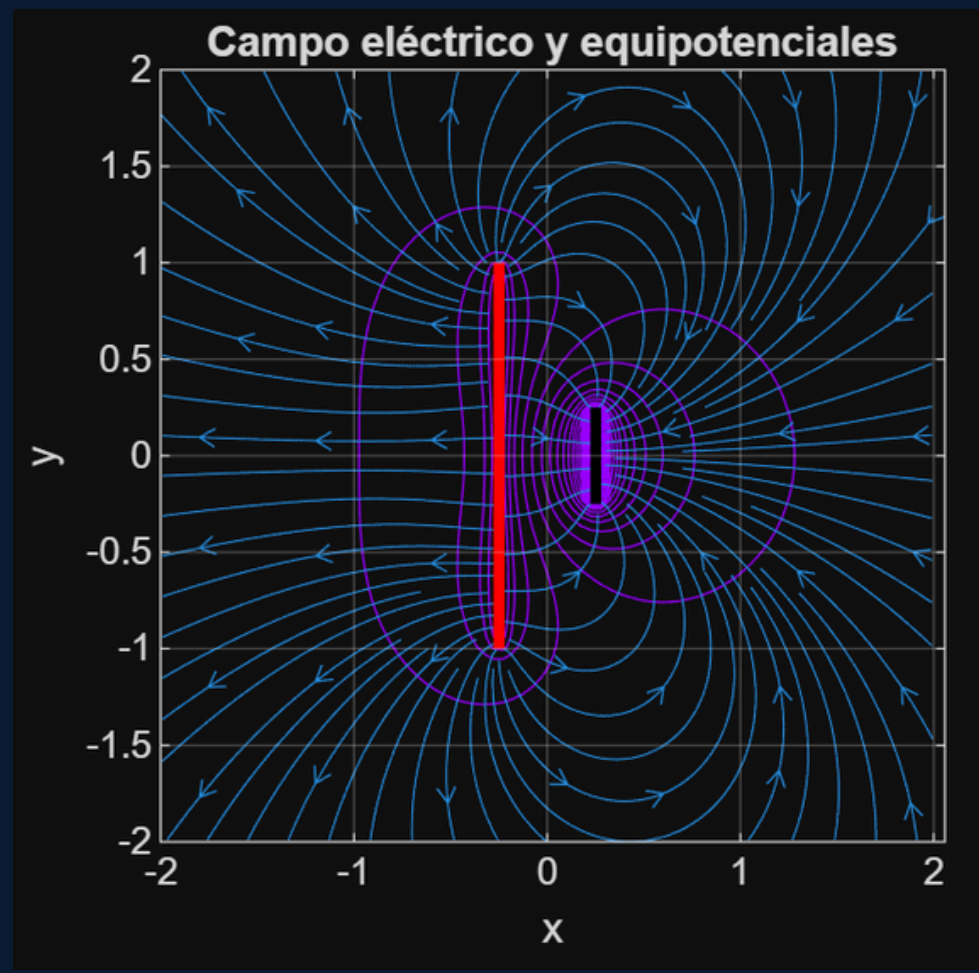
plot(x_val_p, y_val_p, 'r', 'LineWidth', 3)
plot(x_val_n, y_val_n, 'k', 'LineWidth', 3)

xlabel('x')
ylabel('y')
title('Campo eléctrico y equipotenciales')
axis equal
grid on
hold off
```

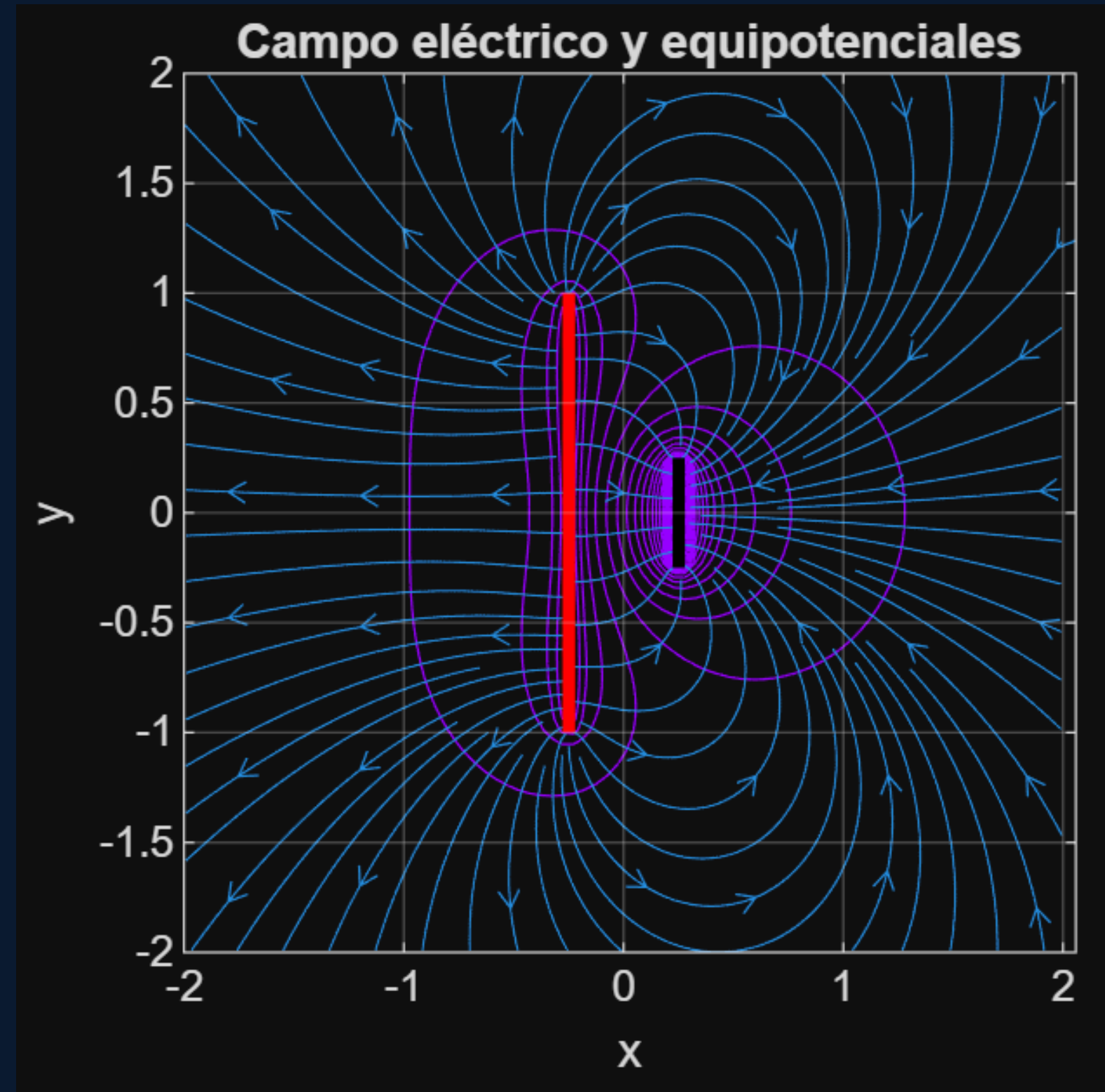
- Graficar líneas

- Dibujar las placas

- Gráfica final



GRÁFICA - Placas Cargadas



MATLAB · Parte 3: Simulación de Glóbulos

Genera el dataset real que entrena los modelos de IA

INFECTADO (clase = 1)

- ▶ $qE_base = 1.4 \times 10^{-6} \text{ C}$
- ▶ $Masa += vol_parásito \times 0.2$
- ▶ Más carga → más deflexión

SANO (clase = 0)

- ▶ $qE_base = 0.8 \times 10^{-6} \text{ C}$
- ▶ Masa $m = 1.0$
- ▶ Menos carga → menos deflexión

Método de Euler

- ▶ Fuerza eléctrica: $F = k_e \cdot dq_n \cdot qE / r^2$ (solo placa negativa)
- ▶ Aceleración: $a_x = F_x/m$, $a_y = F_y/m$
- ▶ Velocidad: $V_x += a_x \cdot dt$, $V_y += a_y \cdot dt$ (dt = 0.02)
- ▶ Posición: $x_e += V_x \cdot dt$, $y_e += V_y \cdot dt$
- ▶ Registra: carga, masa, tiempo de vuelo, velocidad máx, posición final

```
% --- Definir parámetros de simulación ---
nq = 40;      % cantidad de glóbulos simulados
dt = 0.02;    % paso de tiempo
dataset = zeros(nq, 6); % matriz para guardar los datos
```

- Parámetros de simulación

```
% --- Crear figura de simulación ---
figure(2)
```

```
ax_sim = axes;
hold(ax_sim, 'on')
title('Simulación de Separación para Dataset de IA')
xlabel('Posición X')
ylabel('Posición Y')
grid on
axis(ax_sim, [-0.5 1.5 -2 1])
```

- Crear entorno de simulación

```
% --- Dibujar placas cargadas ---
plot(x_val_p, y_val_p, 'r', 'LineWidth', 3)
plot(x_val_n, y_val_n, 'k', 'LineWidth', 3)
```

- Dibujar placas

```
% --- Simular cada glóbulo ---
for q = 1:nq
```

- Iniciar simulación

```
    % --- Asignar si el glóbulo es sano o infectado ---
    if rand() > 0.5
        clase = 0;      % sano
        color = [0 1 0]; % verde
        qE_base = 0.8e-6; % carga base menor
        m = 1;          % masa base
```

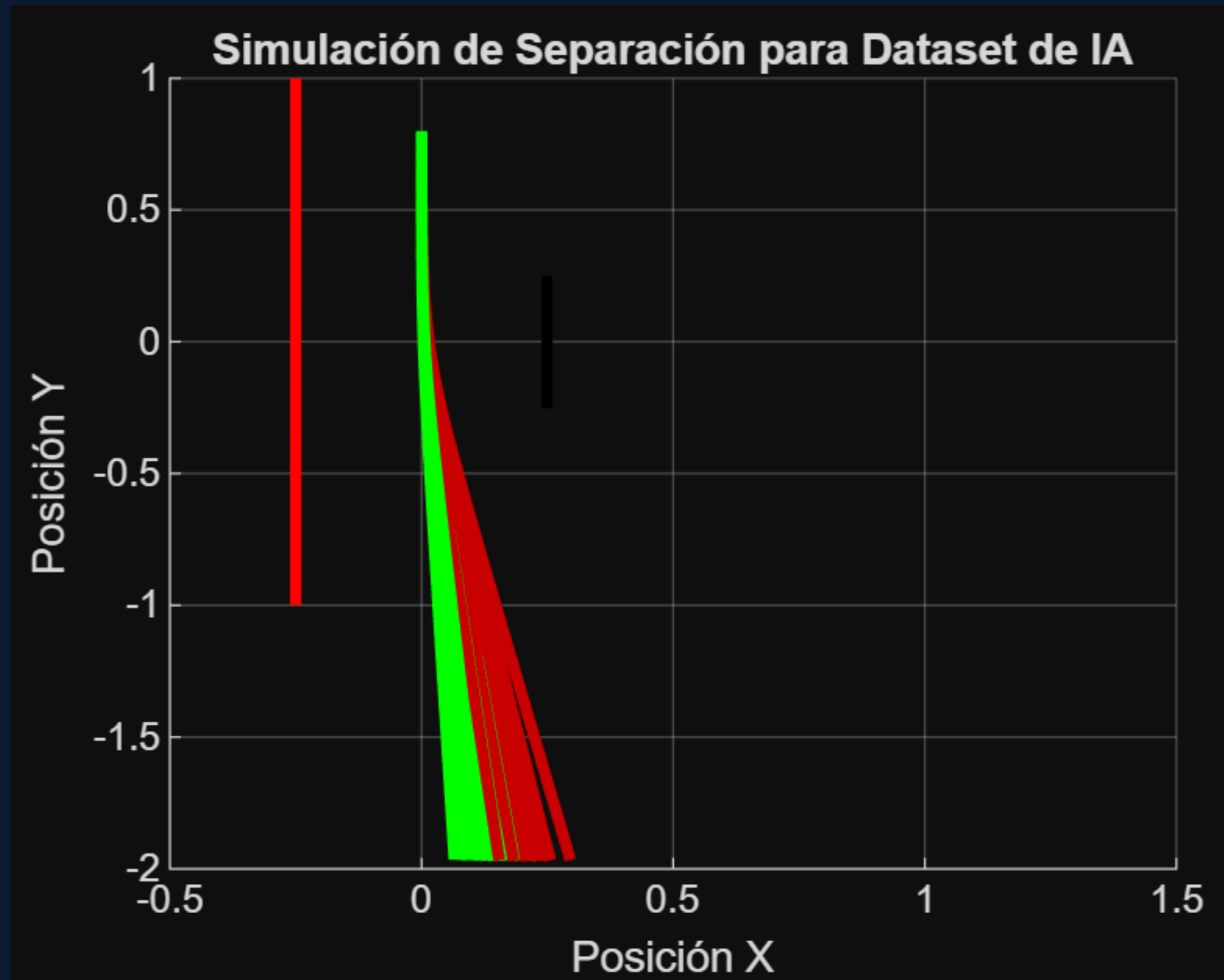
- Clasificar partícula

```
    else
        clase = 1;      % infectado con malaria
        color = [0.8 0 0]; % rojo
        qE_base = 1.4e-6; % carga base mayor
```

- Incrementar masa por infección

```
    % --- Aumentar masa por presencia del parásito ---
    vol_parasito = 0.1 + rand() * 0.8;
    m = 1.0 + (vol_parasito * 0.2);
end
```


GRÁFICA - Simulación de Glóbulos



Dataset Generado · ¿Qué mide cada variable?

Cada fila = 1 glóbulo simulado | 6 columnas | 40 filas

qE Coulombs (C)	m kg (rel.)	t_vuelo segundos	v_max m/s	xe metros	clase 0 / 1
Carga eléctrica • Carga total del glóbulo • Incluye variación aleatoria • SANO $\approx 0.8 \mu\text{C}$ • INFECTADO $\approx 1.4 \mu\text{C}$	Masa • Masa del glóbulo • SANO: $m = 1.0$ • INFECTADO: mayor masa • Depende del parásito	Tiempo de vuelo • Tiempo en simulación • Hasta salir del sistema • Depende de la trayectoria	Velocidad máxima • Pico de velocidad • Durante el recorrido	Posición final X • Posición horizontal final • Indica desviación • Variable clave	Etiqueta (label) • $0 \rightarrow \text{SANO}$ • $1 \rightarrow \text{MALARIA}$ • Variable objetivo

Python · Los 3 Modelos de IA

Entrenan con los datos que MATLAB generó

Preparacion de datos

```
# =====
# CARGA Y PREPARACIÓN DE DATOS
# =====

# Nombres de las columnas del dataset
columnas = ['Carga', 'Masa', 'TiempoVuelo', 'VelMax', 'PosFinalX', 'Clase']
# Cargar archivo CSV
df = pd.read_csv('dataset_celulas.csv', names=columnas)

# Variables de entrada (features) y variable objetivo (label)
X = df[['Carga', 'Masa', 'TiempoVuelo', 'VelMax', 'PosFinalX']]
y = df['Clase']

# =====
# NORMALIZACIÓN DE DATOS
# =====
# Se escalan los datos para que todas las variables estén en la misma escala
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

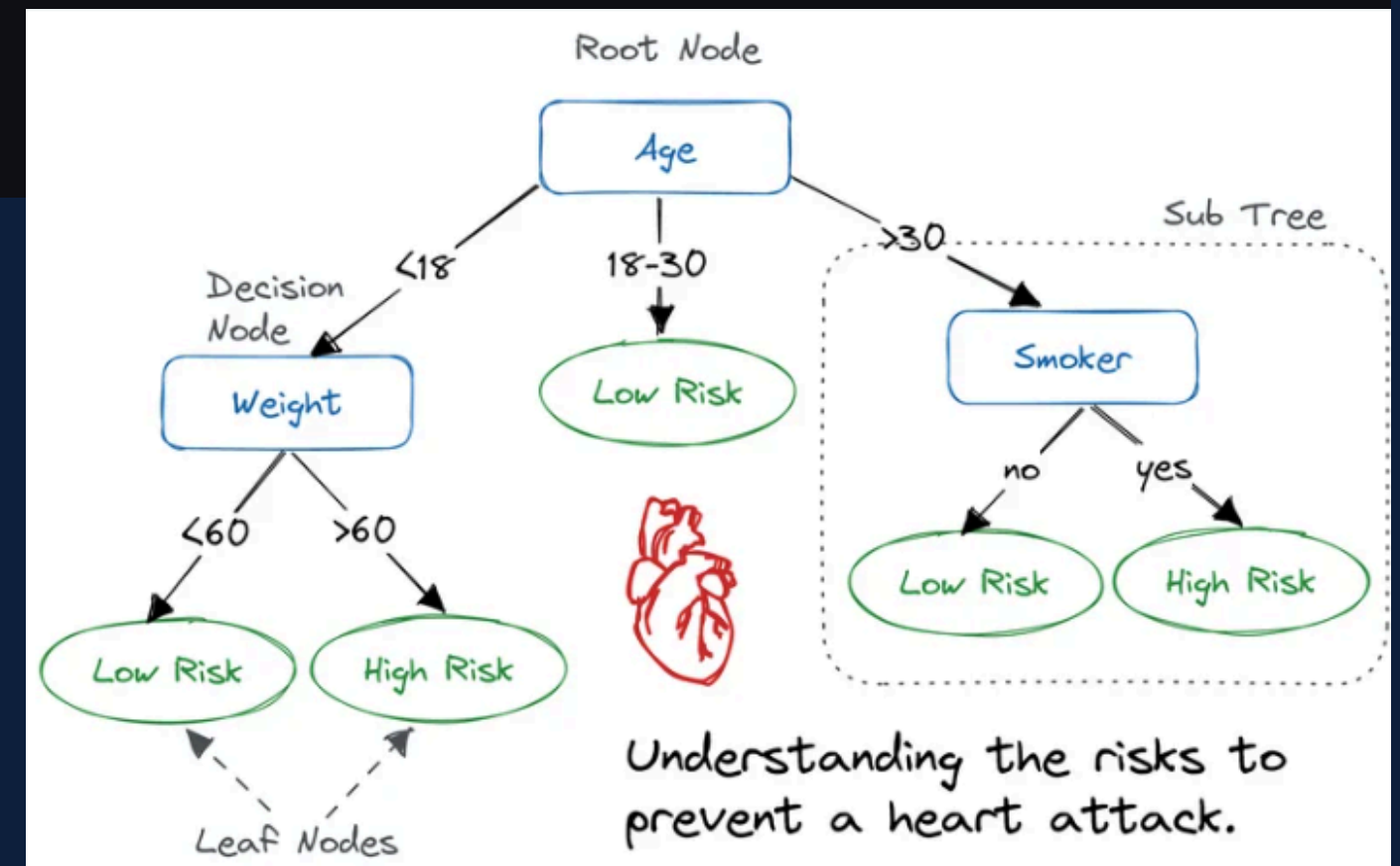
# =====
# DIVISIÓN DE DATOS
# =====
# 70% entrenamiento, 30% prueba
✓ X_train, X_test, y_train, y_test = train_test_split(
    |   X_scaled, y, test_size=0.3, random_state=42 # es una semillaLos modelos tienen componentes aleatorios (como el orden de los datos o el peso inicial de las neuronas)
    )
```


Python · Los 3 Modelos de IA

Entrenan con los datos que MATLAB generó

Modelo 1: Decision Tree

```
# =====  
# MODELO 1: DECISION TREE  
# =====  
# Árbol de decisión que clasifica mediante reglas (if-else)  
  
# Configuración de hiperparámetros para evitar sobreajuste (profundidad limitada)  
  
✓ dtree = DecisionTreeClassifier(  
    criterion='entropy',    # Método de división La entropía calcula cuánta "incertidumbre" hay en un nodo. El árbol elige la división que más reduzca esa incertidumbre  
    max_depth=3,           # Profundidad máxima del árbol, si es pequeño es simple y generaliza, si es grande puede sobreajustar o memorizar  
    min_samples_split=5,   # Mínimo de muestras para dividir un nodo Evita que el árbol pierda tiempo analizando grupos ya muy pequeños.  
    min_samples_leaf=2,    # Mínimo de muestras en una hoja Asegura que ninguna regla final sea tan específica que solo se aplique a un "individuo" (evita el ruido).  
    random_state=42        # Semilla para reproducibilidad.  
)  
dtree.fit(X_train, y_train)    # Entrenamiento del modelo  
y_pred_dt = dtree.predict(X_test) # Predicciones
```



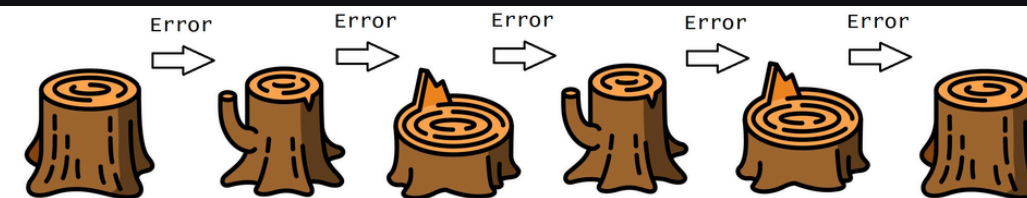
Python · Los 3 Modelos de IA

Entrenan con los datos que MATLAB generó

Modelo 2: Adaboost

```
# =====  
# MODELO 2: ADABOOST  
# =====  
# Ensemble que combina múltiples árboles pequeños para mejorar el rendimiento  
  
ada = AdaBoostClassifier(  
    estimator=DecisionTreeClassifier(max_depth=1), # Árbol débil (stump)  
    n_estimators=50,          # Número de modelos  
    learning_rate=1.0,        # Tasa de aprendizaje,  
    | | | | | | | | | |      # si se sube hace cambios agresivos, si es bajo requiere mas n_estimators pero es estable  
    random_state=42  
)
```

AdaBoost



Tree

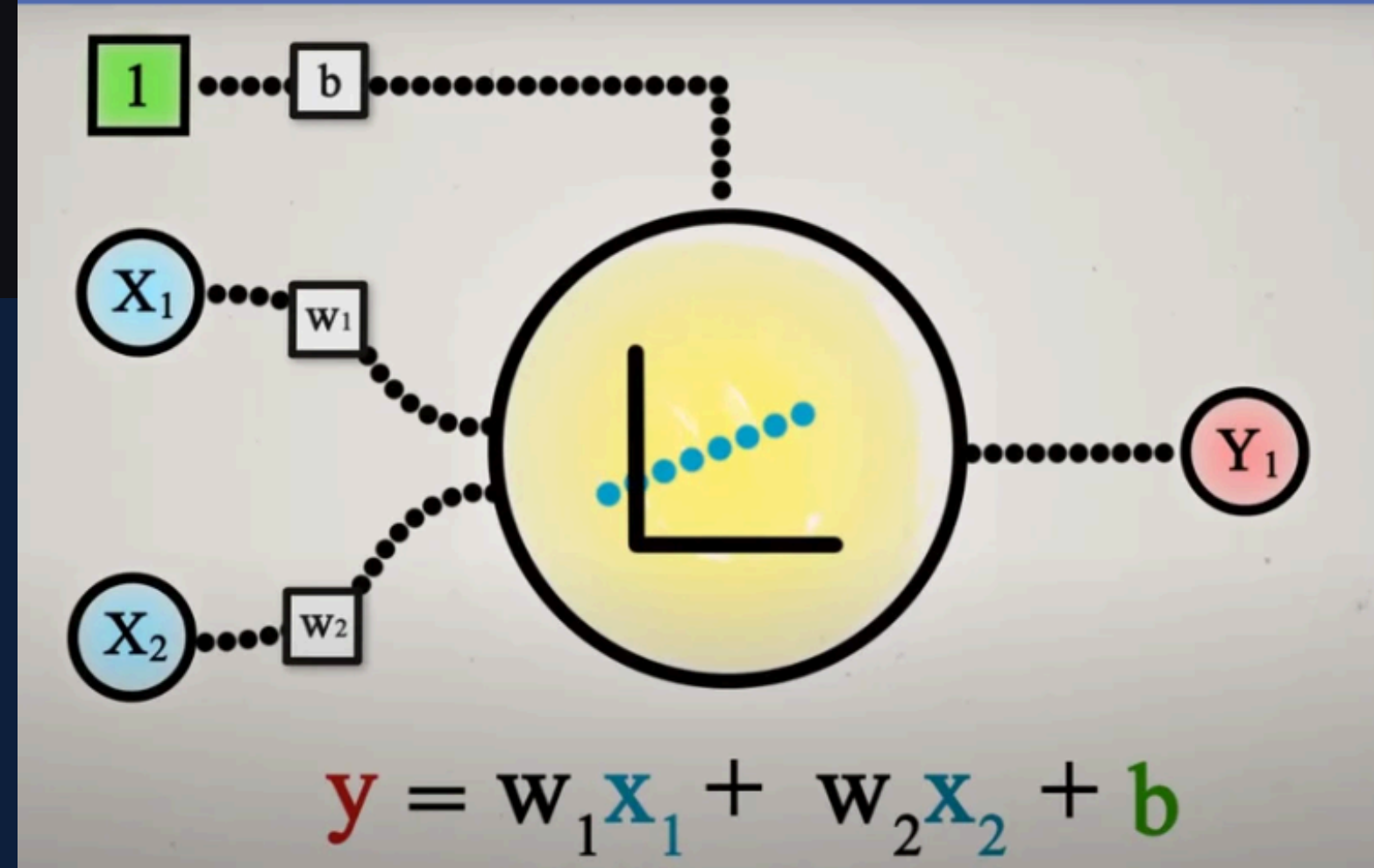


Python · Los 3 Modelos de IA

Entrenan con los datos que MATLAB generó

Modelo 3: Red Neuronal

```
# =====  
# MODELO 3: RED NEURONAL  
# =====  
# Modelo que simula neuronas para aprender relaciones complejas  
  
red_neuronal = MLPClassifier(  
    hidden_layer_sizes=(10,10), # se tienen 5 neuronas al inicio por cada variable. luego dos capas de 10 neuronas, la primera "procesa" y la segunda "refina"  
    activation='relu',          # Función de activación. si es negativo se vuelve 0, si es positivo se queda igual. Esto ayuda a que la red aprenda relaciones no lineales o complejas  
    solver='adam',              # Algoritmo de optimización. primero datos entran, pasan por las capas y predicen, luego regresan para ajustar pesos  
    max_iter=500,               # Iteraciones máximas  
    random_state=42,            # Muestra el entrenamiento en consola  
    verbose=True  
)  
  
print("\n Entrenando Red Neuronal...")  
red_neuronal.fit(X_train, y_train)  
  
y_pred_red = red_neuronal.predict(X_test)
```



Metricas de evaluacion

Para entender las métricas, primero debemos ver de dónde salen los números. El modelo clasifica los datos en cuatro categorías:



$$Accuracy = \frac{VP + VN}{Total\ de\ predicciones}$$

$$Recall = \frac{VP}{VP + FN}$$

Métricas de evaluación

